

TEMA 02 • ESTRATEGIA DEL DATO, ML & DL

Fundamentos de *machine* *learning*.

Conceptos básicos, tipos de aprendizaje y cómo elegir el correcto para cada problema.

ASIGNATURA	Estrategia del Dato, ML y DL
TEMA	02 • Fundamentos de ML
DURACIÓN	120 minutos
SESIÓN	02 / 07

LO QUE VEREMOS HOY

Dos bloques. Una clase. De la definición a saber elegir.

BLOQUE I · 60 MIN · CONCEPTOS BÁSICOS

- ¿Qué es el *machine learning*?
- El perceptrón y la evolución del campo
- IA · ML · Deep Learning, círculos concéntricos
- Tecnologías y comunidad (sklearn, TF, PyTorch)
- Incertidumbre y complejidad de los datos

BLOQUE II · 60 MIN · TIPOS DE APRENDIZAJE

- Supervisado · regresión y clasificación
- Función de pérdida y entrenamiento del modelo
- Ada Lovelace · figura del módulo
- No supervisado y semi-supervisado
- Aprendizaje por refuerzo · MDP
- Práctica final: clasifica el problema

Conceptos básicos de *ML*.

Definición, historia y el lugar del machine learning dentro de la inteligencia artificial.

¿Qué es el *machine learning*?

Una rama de la **inteligencia artificial** donde los sistemas *aprenden patrones a partir de datos* en lugar de seguir reglas escritas a mano.

El programador no codifica la solución: define el problema, aporta los datos, y el algoritmo ajusta sus parámetros para minimizar el error.

CLÁSICA

Programación tradicional

datos + reglas → resultado
El humano escribe las reglas.

ML

Machine learning

datos + resultados → reglas
El modelo descubre las reglas que los explican.

Setenta años de *aprendizaje automático*.

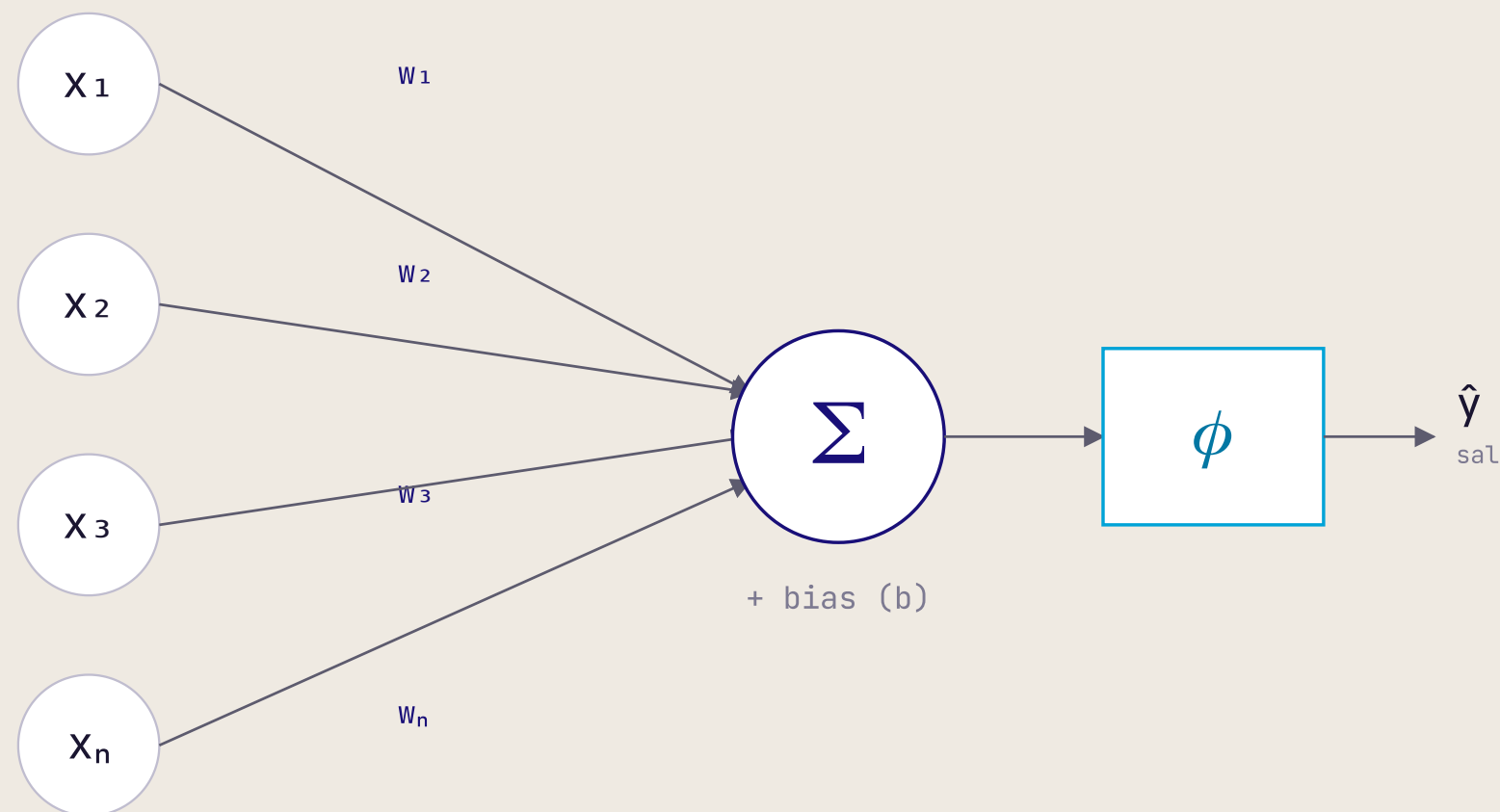
1957	1986	1995	2012	2017	2020+
● Perceptrón Rosenblatt propone la primera neurona artificial entrenable.	● Backpropagation Rumelhart, Hinton, Williams: redes multicapa entrenables.	● SVM & ensembles Auge del ML estadístico clásico (Cortes & Vapnik).	● AlexNet GPU + datos masivos: deep learning gana ImageNet.	● Transformers «Attention is all you need» reordena la NLP moderna.	● Modelos fundacionales LLMs y multimodales a escala industrial.

El ML no nace en 2012: lleva décadas afinándose. Lo que cambia en los 2010 es la combinación de datos, hardware y software abierto.

2.2 · EL PERCEPTRÓN

El *perceptrón*.

La unidad mínima a partir de la cual se construye todo lo demás.

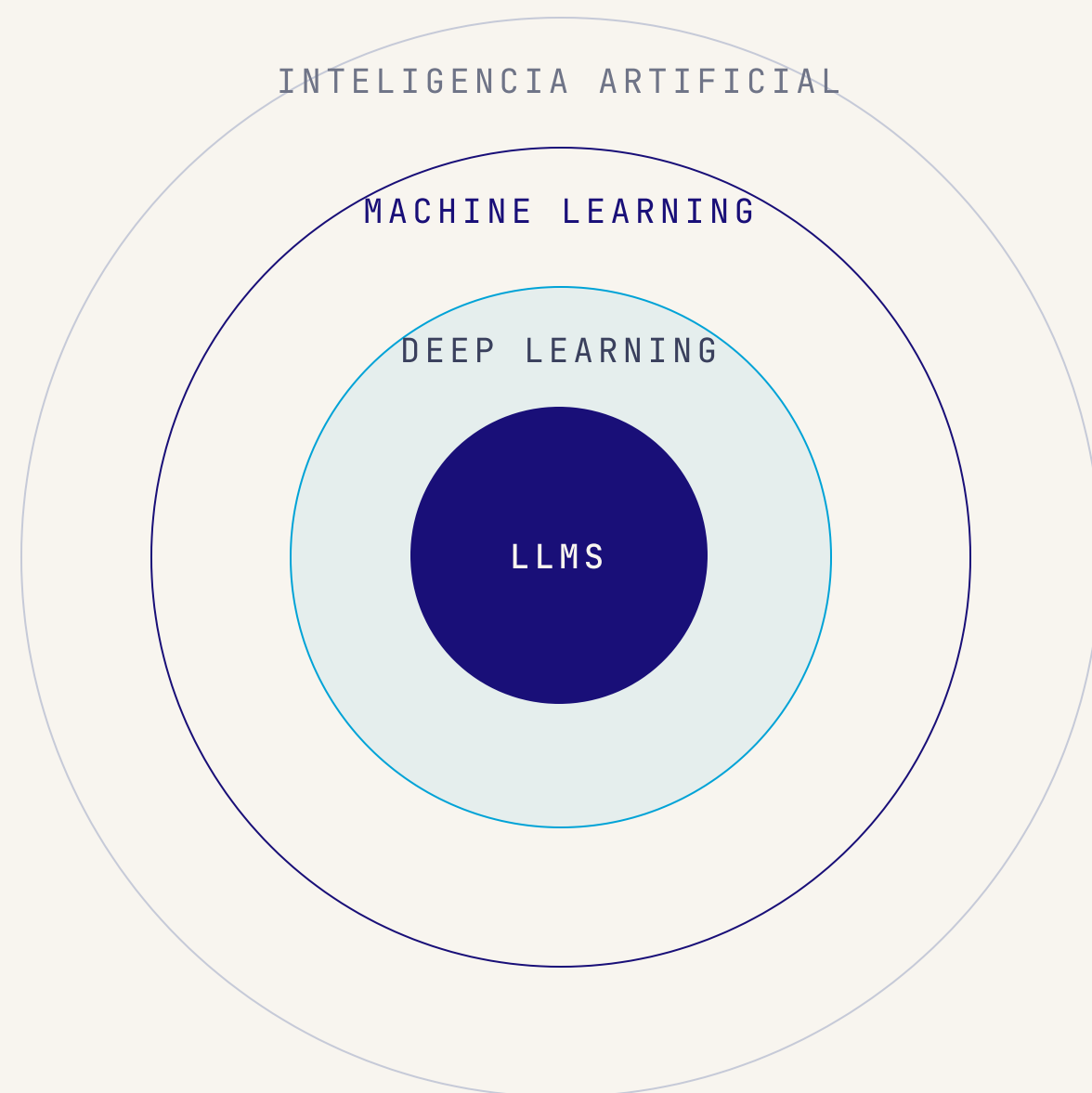


Cada entrada x_i se multiplica por un peso w_i . El perceptrón los suma, aplica una función de activación ϕ y produce una decisión.

$$\hat{y} = \phi(w \cdot x + b)$$

Aprender = encontrar pesos w y sesgo b que minimizan el error sobre los datos. Toda red neuronal es una pila de estas unidades.

Inteligencia artificial, *ML* y deep learning.



IA

Cualquier sistema que imita capacidades cognitivas

Reglas expertas, búsqueda, planificación, ML.

ML

Subconjunto que aprende de datos

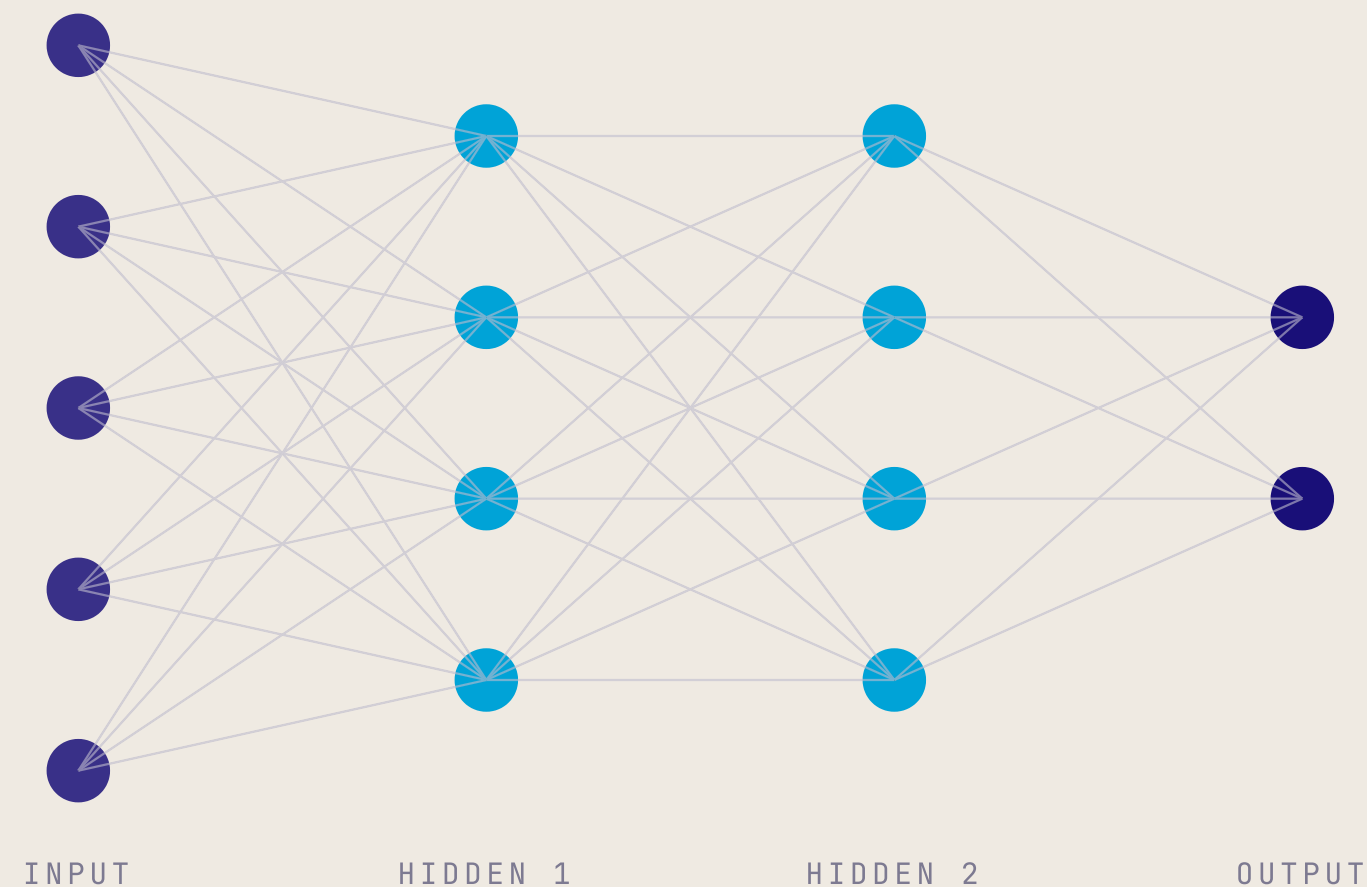
Regresión, árboles, SVM, redes, clustering...

DL

ML con redes neuronales profundas

Visión, lenguaje, audio. Aprende representaciones.

Deep learning y redes neuronales profundas.



Apila *muchas capas de perceptrones* conectados. Cada capa transforma los datos y aprende una representación más abstracta que la anterior.

- Aprende **características** automáticamente.
- Brilla con datos no estructurados: imagen, texto, audio.
- Necesita más datos y más cómputo que el ML clásico.

Machine learning *vs.* deep learning.

DIMENSIÓN	ML CLÁSICO	DEEP LEARNING
Datos necesarios	Cientos / miles de filas.	Cientos de miles o millones.
Características	El humano diseña features.	Las aprende la red, capa a capa.
Hardware	CPU normal suele bastar.	GPU/TPU para tiempos razonables.
Interpretabilidad	Más fácil de explicar.	Caja negra, requiere XAI.
Tipo de dato fuerte	Tabular y estructurado.	Imagen, texto, audio, vídeo.
Algoritmos típicos	Regresión, SVM, random forest, k-means.	CNN, RNN, transformers, GANs.

Las herramientas con las que *vamos a trabajar*.

Python ha ganado la guerra. El resto del stack se organiza en torno a él.

ML CLÁSICO

scikit-learn

API homogénea para regresión, clasificación, clustering. Punto de entrada obligado.

DL · GOOGLE

TensorFlow

Producción a escala, despliegue móvil/embebido (TF Lite, TF Serving).

DL · META

PyTorch

Estándar en investigación. Grafos dinámicos, depuración natural en Python.

ALTO NIVEL

Keras

API amigable sobre TF / JAX / PyTorch. Ideal para prototipar rápido.

NUMPY

PANDAS

MATPLOTLIB

JUPYTER

HUGGING FACE

KAGGLE

PAPERS WITH CODE

ARXIV

El ML vive en la *incertidumbre*.

No predice verdades; estima la respuesta más probable bajo datos imperfectos.

σ

Datos ruidosos

Sensores, errores humanos, valores ausentes. El modelo debe ser robusto al ruido.

∂

Mundo no estacionario

Las distribuciones cambian con el tiempo. Lo de ayer puede no funcionar mañana.

∞

Espacios enormes

Pocos ejemplos frente a billones de combinaciones posibles. Hay que generalizar.

Por eso la *estrategia del dato* de la sesión 1 es la base de todo: sin datos limpios, representativos y bien comprendidos, ningún algoritmo nos salva.

Tipos de *aprendizaje.*

Supervisado, no supervisado, semi-supervisado
y por refuerzo. El modelo no cambia: cambia
cómo aprende.

Cómo puede *aprender un modelo*.

01

Supervisado

Datos con etiqueta. El modelo aprende a reproducir la respuesta correcta.

02

No supervisado

Datos sin etiqueta. Busca estructura y patrones latentes.

03

Semi-supervisado

Mezcla: pocos datos etiquetados y muchos sin etiquetar.

04

Por refuerzo

Aprende por prueba y error usando recompensas y castigos.

Importante: el tipo de aprendizaje no es el modelo. Un mismo problema puede abordarse con varios algoritmos, y un mismo algoritmo (p. ej. una red neuronal) puede entrenarse de varias maneras.

Aprender con la *respuesta delante*.

Cada ejemplo de entrenamiento viene con su salida correcta.

El dataset incluye una columna especial — la *etiqueta* o *target* — con el valor que queremos predecir. El modelo ajusta sus parámetros para que su predicción se acerque cada vez más a esa etiqueta.

- ¿Es spam o no? · **clasificación binaria**
- ¿Qué dígito hay en esta imagen? · **multiclase**
- ¿Cuánto valdrá esta casa? · **regresión**
- ¿Cuántos clientes vendrán mañana? · **regresión**

NECESITA

Un dataset etiquetado

Construirlo es lento y caro: anotación humana, encuestas, sensores con ground truth.

DEVUELVE

Una función $f(x) \approx y$

Aprendida a partir de ejemplos (x_i, y_i) . Ya entrenada, predice y para una x nueva.

Las columnas que *entran...* y la que *sale*.

ID	M²	HABITACIONES	BAÑOS	ANTIGÜEDAD	BARRIO	PRECIO (€) · TARGET
01	62	2	1	35	Tetuán	198 000
02	85	3	2	12	Salamanca	410 000
03	120	4	2	5	Chamberí	685 000
04	48	1	1	60	Lavapiés	175 000
05	95	3	2	22	Retiro	520 000
06	150	5	3	3	Chamberí	920 000
...	...	...	...	...	...	...

FEATURES (X)

Lo que el modelo recibe

Las columnas de entrada que combina para hacer su predicción.

TARGET (y)

Lo que el modelo aprende

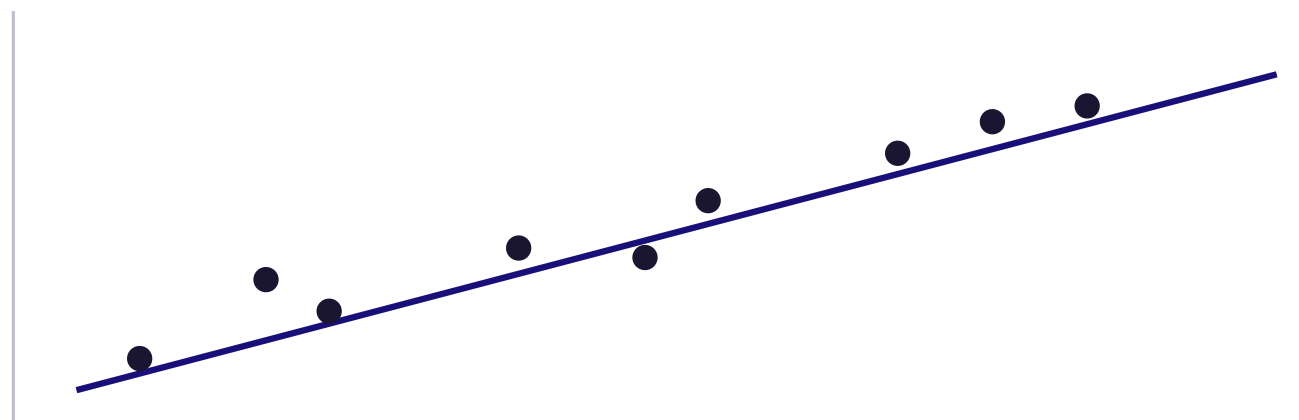
La columna que el modelo aprende a reproducir.
Disponible en entrenamiento; no, en producción.

Sin el target no hay aprendizaje supervisado.
Construirlo es a menudo el cuello de botella del proyecto.

Regresión *vs.* clasificación.

REGRESIÓN

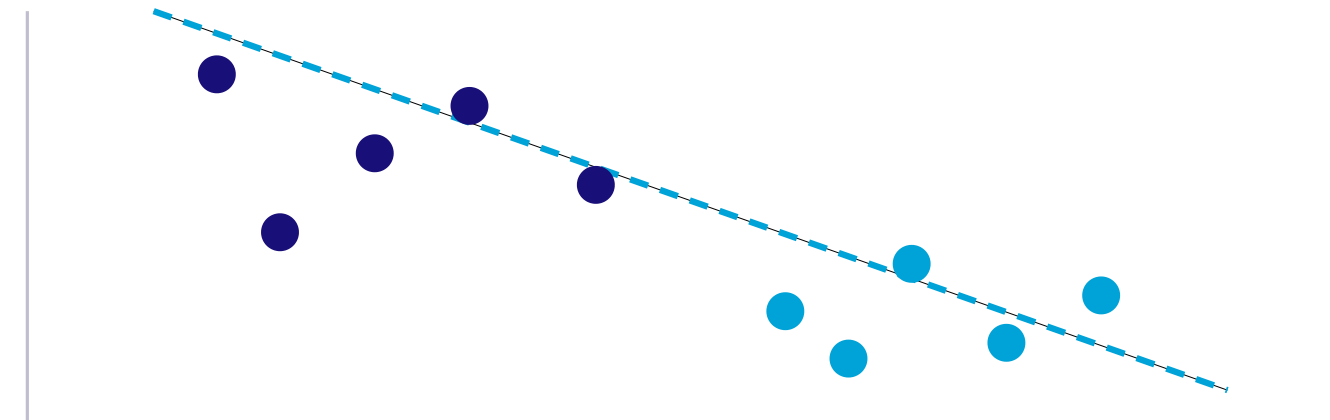
Predice un *valor continuo*



- Precio de una vivienda
- Demanda semanal de stock

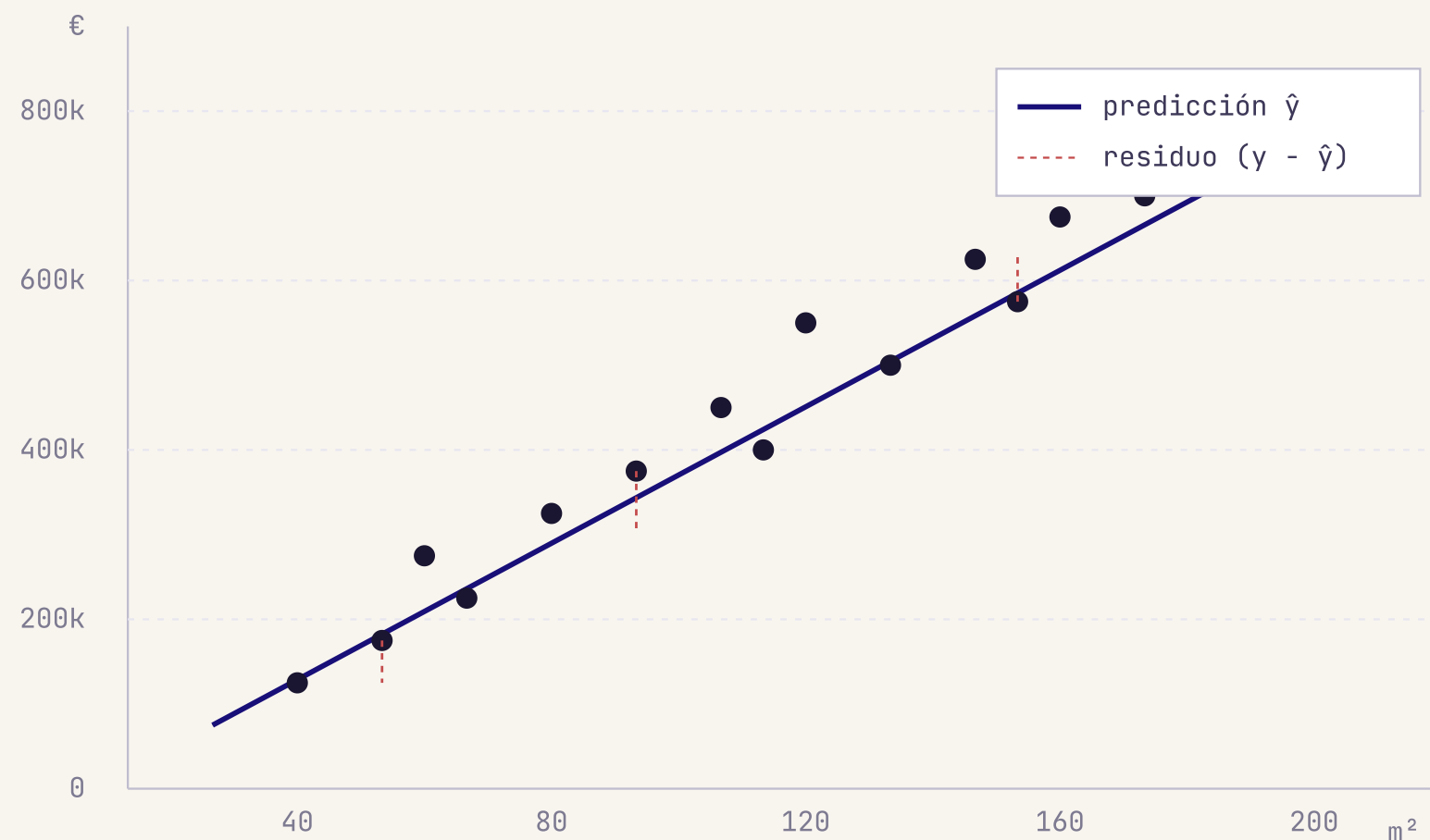
CLASIFICACIÓN

Predice una *categoría*



- Spam / no spam
- Diagnóstico: sano / enfermo

Predecir el precio de una vivienda según sus m^2 .



Cada punto es una vivienda real. La *línea índigo* es la predicción del modelo.

La distancia vertical de cada punto a la recta — el **residuo** — mide cuánto se equivoca el modelo en ese ejemplo.

$$\hat{y} = w \cdot m^2 + b$$

Aprender = encontrar w y b que hagan los residuos lo más pequeños posible globalmente.

El modelo aprende *bajando una montaña a ciegas*.

1 EMPIEZA ARRIBA DEL TODO

El modelo hace su **primera predicción aleatoria**. Por ejemplo: "todos los pisos valen 50.000 €". Evidentemente, se equivoca mucho.

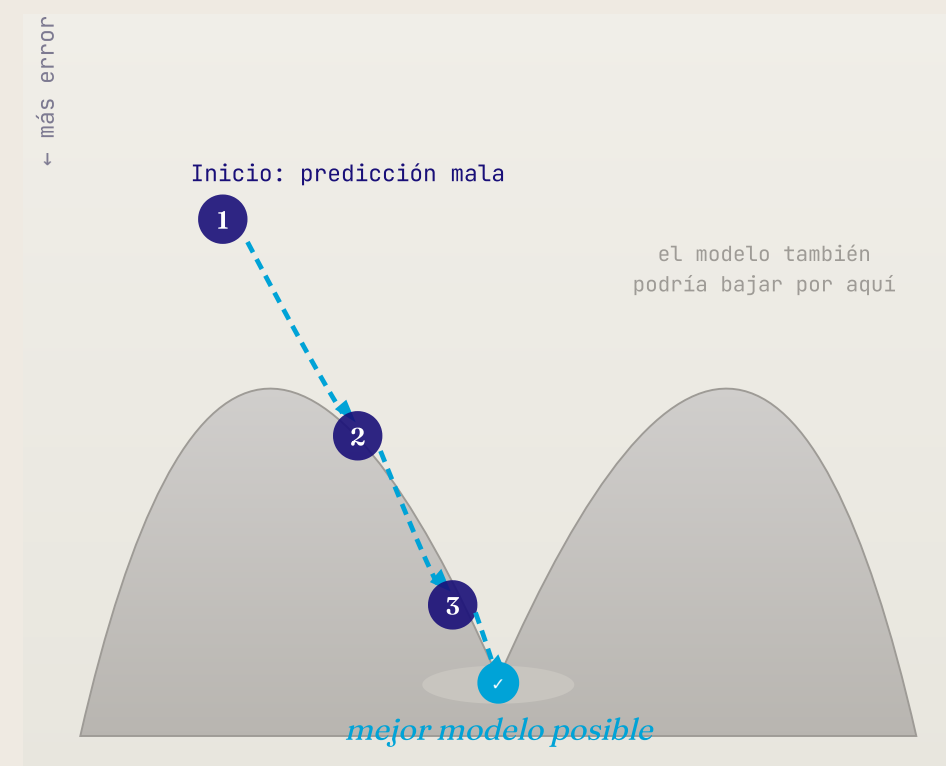
2 MIDE EL ERROR TOTAL

Compara sus respuestas con los precios reales. **Cuanto más alto en la montaña, más se ha equivocado**. Este número es el "error del modelo".

3 DA UN PASO CUESTA ABAJO

Ajusta un poquito sus predicciones en la dirección que **reduce el error**. Repite miles de veces hasta llegar al valle.

↑ El valle = el mejor modelo posible. Cuando ya no puede bajar más, ha terminado de aprender.



Cada paso = ajuste de predicciones · miles de iteraciones

Pipeline de *entrenamiento supervisado*.



Es un *proceso iterativo*: rara vez sale bien a la primera. Se vuelve sobre los datos, sobre el modelo o sobre los hiperparámetros hasta alcanzar el rendimiento objetivo.

Train · Validation · *Test*.

Tres conjuntos disjuntos. Tres roles distintos.

TRAIN · 70%

VAL · 15%

TEST · 15%

CONJUNTO DE ENTRENAMIENTO

El modelo lo ve y se ajusta

Se usa para calcular la pérdida y actualizar los parámetros. La mayoría de los datos.

CONJUNTO DE VALIDACIÓN

Para elegir entre modelos

Se usa para comparar configuraciones e hiperparámetros. El modelo no se entrena con él.

CONJUNTO DE PRUEBA

Examen final, una sola vez

Solo se toca al final, para reportar el rendimiento real esperado en producción.

Mezclarlos = trampa. Si el modelo «ve» el test durante el ajuste, las métricas mienten.

2.3 · ¿QUÉ PASA DESPUÉS DE ENTRENAR?

El modelo aprendió. Ahora solo necesita *la pregunta*.

1

ENTRENAMIENTO

El modelo estudia miles de ejemplos

Le mostramos pisos con sus precios reales. Aprende qué características influyen más en el valor.

60 m ²	→	180.000 €
90 m ²	→	260.000 €
120 m ²	→	350.000 €

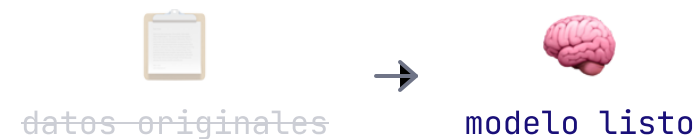


2

EL MODELO QUEDA LISTO

Ya no necesita los ejemplos originales

Es como un experto que ha estudiado mucho: ya sabe lo que sabe. No necesita releer sus apuntes cada vez que alguien le pregunta.



3

PREDICCIÓN

Haces una pregunta nueva y responde

Le das un piso que nunca ha visto y te dice cuánto cree que vale.

75 m² ← nuevo
→ el modelo calcula...
≈ 218.000 €

Medir, comparar, ajustar.

MÉTRICAS TÍPICAS

Regresión	MAE · RMSE · R ²
Clasificación	Accuracy · Precision · Recall · F1
Multiclase / desbalanceo	Confusion matrix · ROC-AUC
Ranking / búsqueda	NDCG · MAP

Elegir bien la métrica importa tanto como elegir el modelo: optimiza lo que vas a evaluar.

AJUSTES POSIBLES

- **Hiperparámetros** · learning rate, profundidad, regularización.
- **Más / mejores datos** · ampliar, limpiar, balancear clases.
- **Otras features** · selección, ingeniería, escalado.
- **Otro modelo** · cambiar la familia si se ha saturado.
- **Regularización** · evitar el sobreajuste.

Cuidado con el sobreajuste: un modelo perfecto en train y mediocre en test ha *memorizado* en lugar de *generalizar*.

PIONEROS DE LA IA • 01

Ada *Lovelace*.

AUGUSTA ADA KING • CONDESA DE LOVELACE • 1815–1852 • LONDRES

La primera persona que vio que una máquina podía hacer *algo más que cuentas*.

Hija del poeta Lord Byron, su madre la educó a propósito en matemáticas para alejarla de la imaginación paterna. El resultado fue una mente única que combinaba *rigor analítico* con lo que ella llamaba «**ciencia poética**». A los 17 años conoció a Charles Babbage y quedó fascinada con su *máquina analítica*, una computadora mecánica que nunca llegó a construirse.

En 1843 tradujo del italiano un artículo sobre la máquina y le añadió siete notas propias —tres veces más largas que el original— que incluían el primer algoritmo publicado de la historia: un método para calcular **números de Bernoulli**. En la *Nota G* describe iteraciones, variables y bifurcaciones casi un siglo antes del primer ordenador real.

Su salto visionario: si la máquina podía manipular *símbolos*, no solo números, también podría componer música o razonar. Anticipó el ML moderno... y al mismo tiempo formuló la «*objeción de Lady Lovelace*»: una máquina solo hace lo que se le ordena. El debate sigue vivo hoy.

1843 Notas a la máquina analítica • primer algoritmo publicado

LEGADO Lenguaje «Ada» del Departamento de Defensa de EE. UU. (1980)

HOY Ada Lovelace Day • cada 2.º martes de octubre

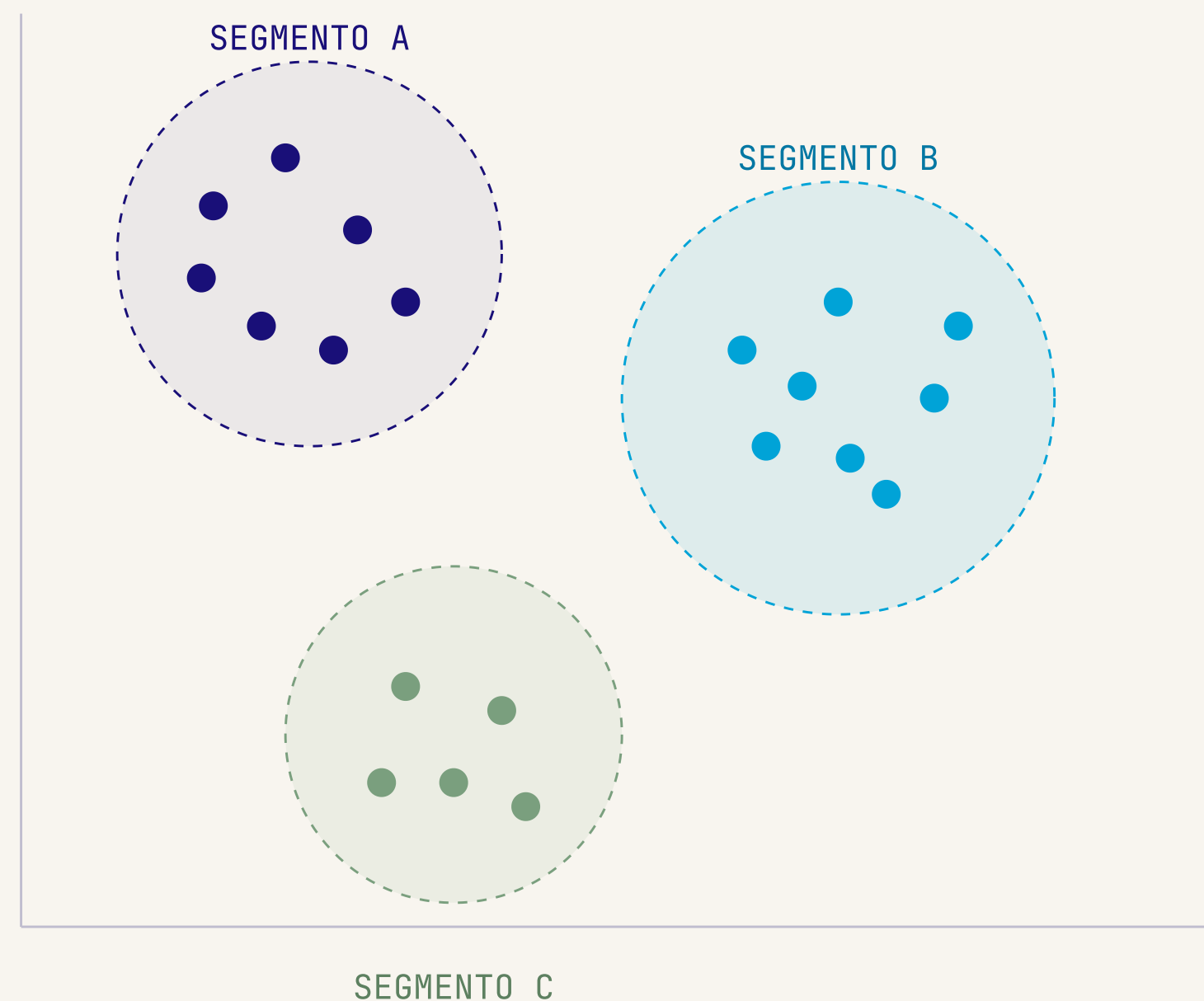


Encontrar estructura cuando *no hay etiquetas*.

Los datos no traen target. El modelo busca *patrones latentes* en los datos: agrupaciones, asociaciones, dimensiones útiles, anomalías.

En el mundo real, la mayoría de los datos llegan así — crudos, sin anotar. El no supervisado es la puerta de entrada a esos datos.

- No requiere etiquetado humano · escala mejor.
- Útil cuando ni siquiera sabemos qué buscar.
- La validación es más cualitativa que numérica.
- Suele ser **el primer paso** antes de un proyecto supervisado.



Segmentar usuarios para *servir el anuncio correcto.*

Una red social tiene millones de perfiles. Nadie los ha etiquetado. ¿Cómo decidimos qué anuncio mostrar a cada uno?

DATOS CRUDOS DISPONIBLES

- Edad declarada, ubicación, idioma.
- Patrones de actividad: horas, frecuencia, dispositivos.
- Temas que sigue y con los que interactúa.
- Tipo de contenido que crea (texto, foto, vídeo).
- Red social: con quién interactúa y cómo.

El algoritmo encuentra grupos coherentes *sin que nadie le diga qué buscar*. Cada grupo se traduce después en una estrategia de campañas.

CLUSTER 01

Jóvenes urbanos

18—28 · móvil · vídeo corto · noche.

CLUSTER 02

Profesionales activos

28—45 · escritorio · noticias · mañanas.

CLUSTER 03

Familias

30—50 · fotos · grupos · fines de semana.

CLUSTER 04

Aficionados de nicho

edad mixta · alta interacción en pocos temas.

CLUSTER 05

Pasivos / lurkers

consumo alto · interacción mínima.

CLUSTER 06

Creadores

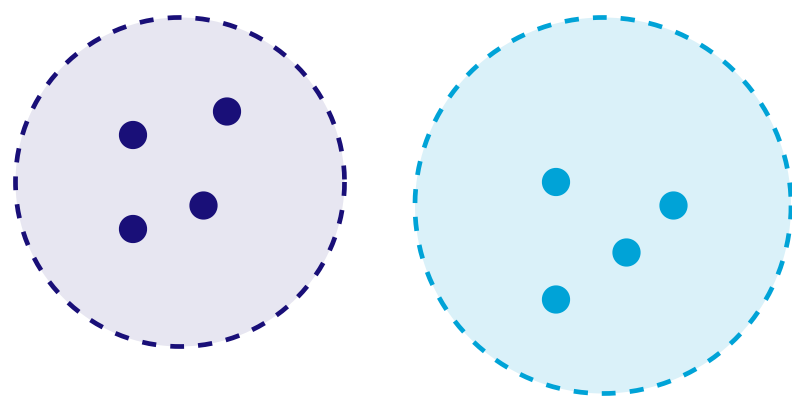
publican mucho · audiencia propia.

Tres preguntas, *tres familias*.

01 · AGRUPAR

Clustering

«¿Qué grupos naturales hay aquí dentro?»

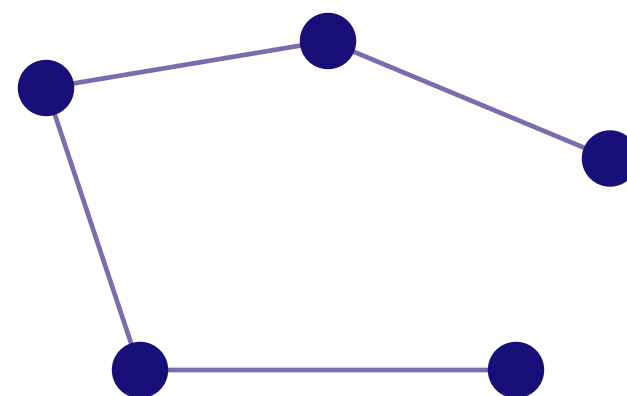


k-means · DBSCAN · jerárquico · GMM

02 · RELACIONAR

Asociación

«¿Qué cosas tienden a aparecer juntas?»

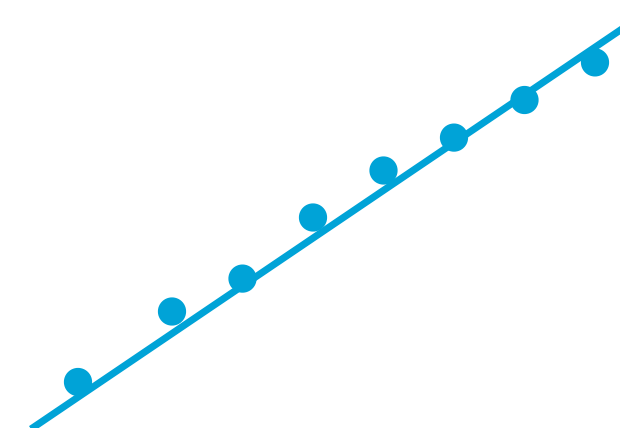


apriori · FP-growth · eclat

03 · COMPRIMIR

Reducción de dimensionalidad

«¿Qué ejes capturan la información esencial?»

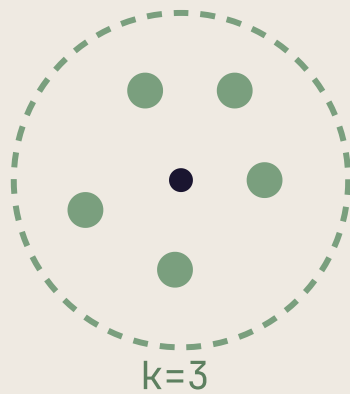
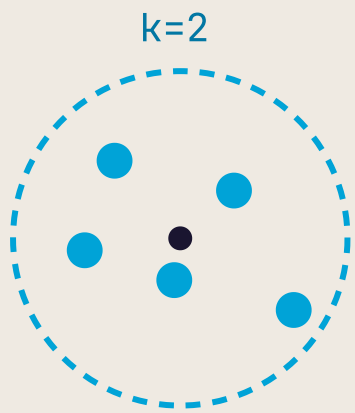
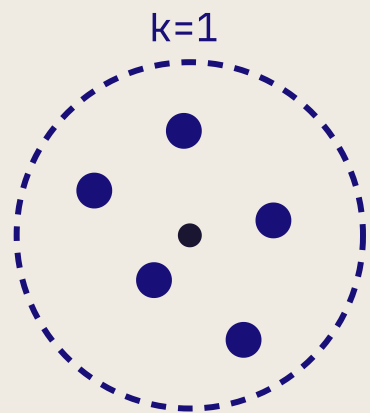


PCA · t-SNE · UMAP · autoencoders

Agrupar por *similitud*.

Las observaciones de un mismo grupo se parecen entre sí; las de grupos distintos, no.

k-means · iterar centroide → reasignar → repetir



ALGORITMOS TÍPICOS	
k-means	Centroides · k fijado a priori · clusters esféricos.
DBSCAN	Por densidad · descubre k · separa ruido.
Jerárquico	Construye un dendrograma · cortas a la altura que quieras.
GMM	Mezcla de gaussianas · pertenencia probabilística.

Aplicaciones: segmentación de clientes, análisis de redes sociales, agrupación de documentos, detección de comunidades.

«Si A, entonces probablemente *B*.»

Descubrir reglas en grandes conjuntos de transacciones. El caso clásico: análisis de la cesta de la compra.

REGLAS QUE EL MODELO DESCUBRE

{pañales} → {cerveza}	conf 0.78
{pan, mantequilla} → {leche}	conf 0.91
{iPhone} → {funda + cargador}	conf 0.66
{viaje + niños} → {seguro familiar}	conf 0.54

Métricas: *support* (frecuencia conjunta), *confidence* (probabilidad condicional) y *lift* (cuánto mejora la regla frente al azar).

APLICACIONES

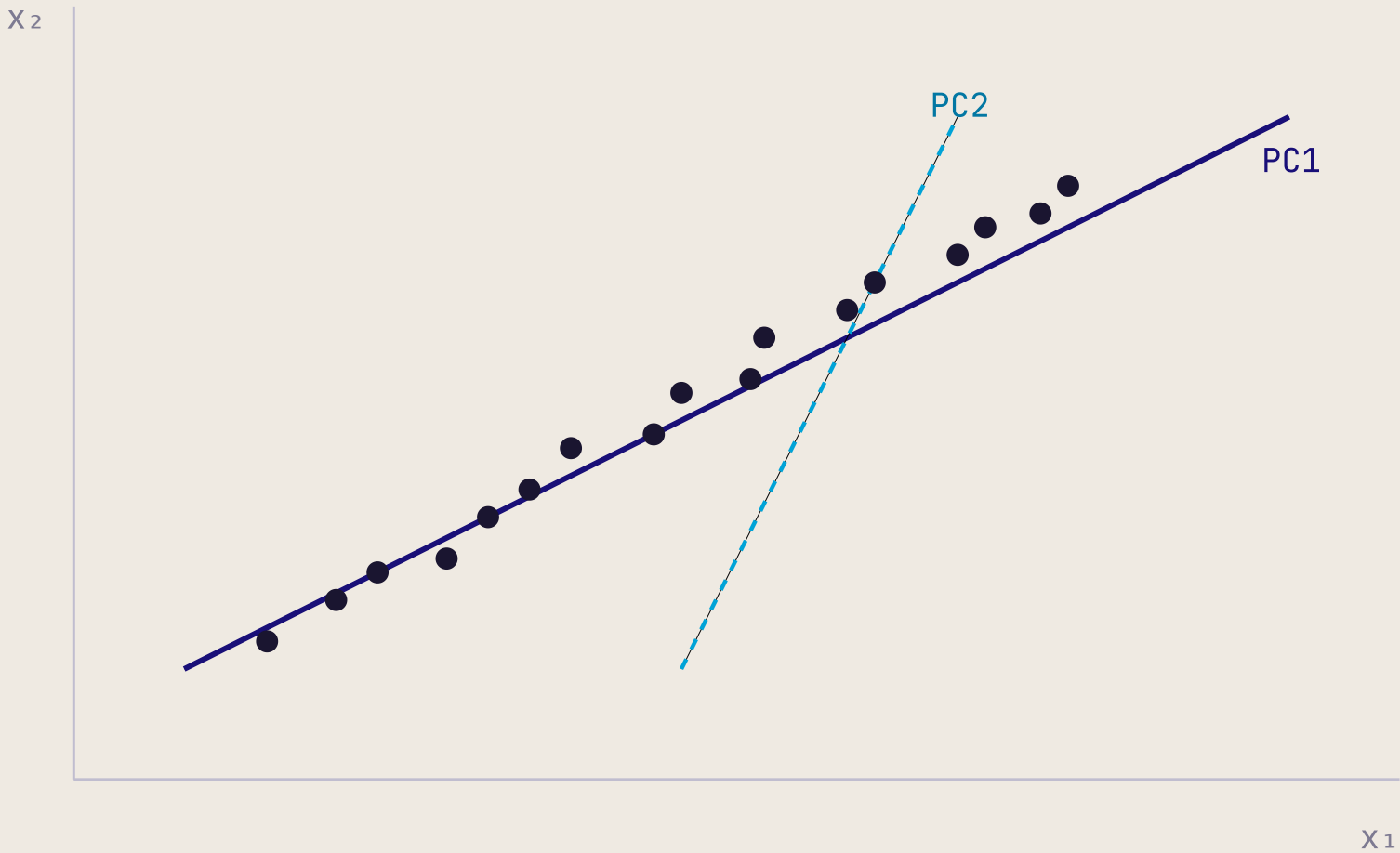
- Cesta de la compra y colocación de productos.
- Recomendación: «quien compró X también compró Y».
- Análisis de logs y secuencias de uso.
- Detección de patrones en historiales clínicos.
- Cross-selling y bundling.

Algoritmos: *Apriori* recorta combinaciones por podas sucesivas; *FP-growth* usa un árbol compacto y es más rápido en datasets grandes.

Comprimir *sin perder lo importante.*

Pasar de cientos de variables a 2 o 3 dimensiones interpretables. Útil para visualizar y para acelerar el resto del pipeline.

PCA · proyectar sobre los ejes de máxima varianza



MÉTODOS	
PCA	Lineal · ejes de máxima varianza · rápido y explicable.
t-SNE	No lineal · preserva vecindad local · visualización 2D.
UMAP	Más rápido y preserva mejor la estructura global.
Autoencoders	Red neuronal · compresión aprendida no lineal.

Para qué: visualizar datos de alta dimensión, eliminar redundancia, mitigar la *maldición de la dimensionalidad*, comprimir antes de entrenar.

Donde el *no supervisado* ya está trabajando.

DETECCIÓN DE ANOMALÍAS

Lo que se sale del patrón

Fraude bancario, fallos en sensores industriales, intrusiones en red. El modelo aprende «lo normal» y avisa cuando algo no encaja.

SISTEMAS DE RECOMENDACIÓN

Filtrado colaborativo

Netflix, Spotify, Amazon. Encuentra usuarios o ítems similares sin saber por qué se parecen.

PROCESAMIENTO DE LENGUAJE

Embeddings y temas

Word2Vec, topic modeling (LDA). Descubrir significado y temas a partir de texto crudo.

VISIÓN POR COMPUTADOR

Pre-entrenamiento

Aprendizaje contrastivo y autoencoders sobre millones de imágenes sin etiqueta. Base de los modelos fundacionales.

BIOINFORMÁTICA

Subtipos celulares

Agrupar células según expresión génica. Descubrir subtipos de tumor, identificar tejidos.

SEGURIDAD Y FRAUDE

Comportamiento atípico

Logs de acceso, transacciones, telemetría. El modelo no necesita ejemplos previos de ataque.

EDA AUTOMÁTICO

Explorar datasets nuevos

Antes de modelar nada, clustering + PCA dan un mapa rápido del terreno.

PROCESAMIENTO DE GRAFOS

Detección de comunidades

Redes sociales, fraude organizado, biología de redes. Encontrar grupos sin pedir el target.

Sin *intervención humana directa*.

La gran ventaja del aprendizaje no supervisado: *no hace falta etiquetar*. El modelo trabaja directamente sobre los datos crudos.

Esto cambia la economía del proyecto: lo que limita ya no es el coste humano de anotación, sino la capacidad de cómputo y la calidad de los datos disponibles.

Pero también lo hace más difícil de evaluar: no hay una respuesta correcta contra la que comparar. La validación es **cualitativa, comparativa y dependiente del contexto de negocio**.

VENTAJAS

Escala con los datos

Sin cuello de botella humano. Aprovecha datos crudos. Útil cuando el dominio no está bien definido.

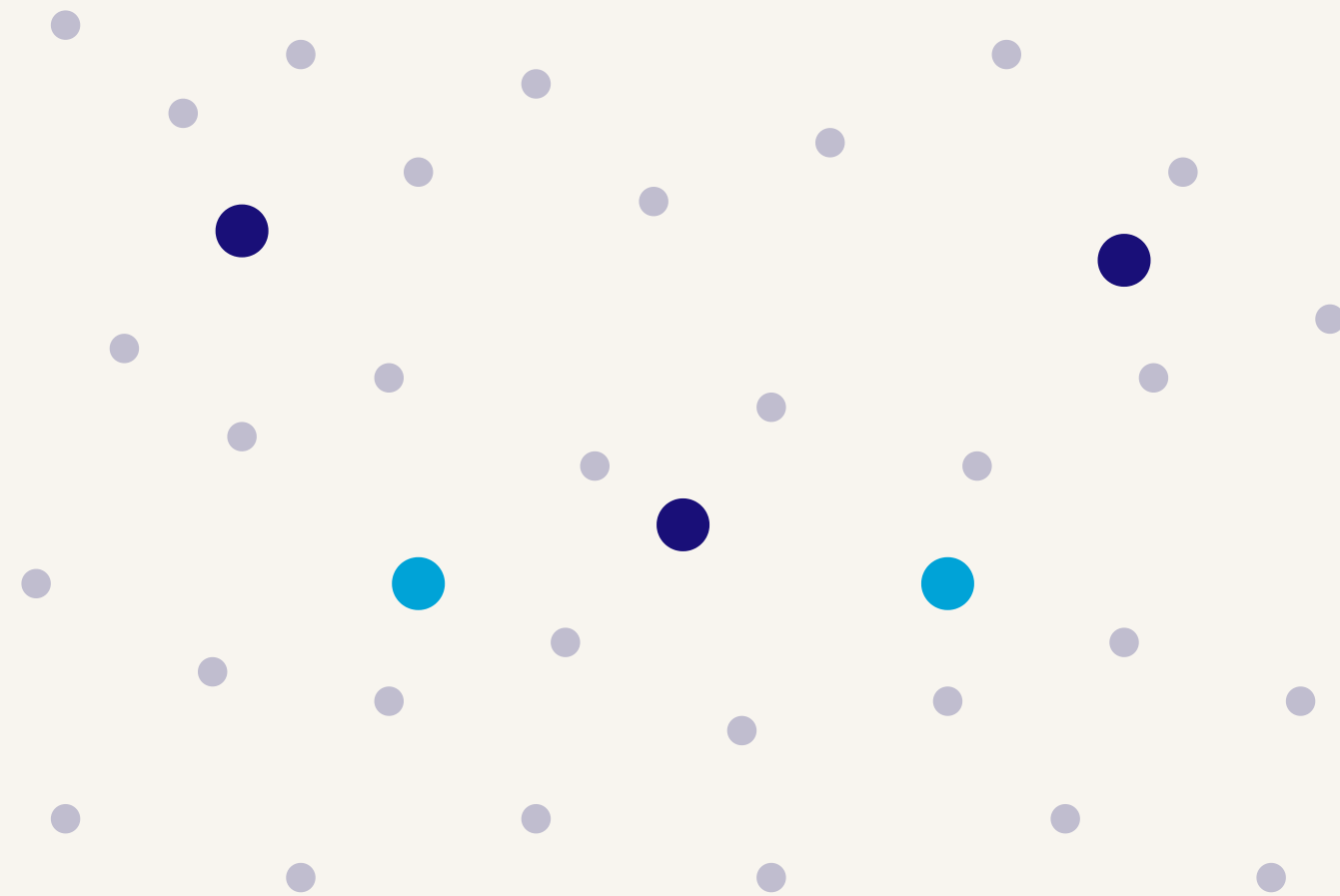
RETOS

Validación más sutil

«¿Estos clusters son útiles?» se responde con expertos del negocio, no con métricas absolutas. Riesgo de patrones espurios.

Semi-supervisado.

Pocos datos etiquetados + muchos sin etiquetar. Lo mejor de los dos mundos.



• pocos etiquetados • muchos sin etiquetar

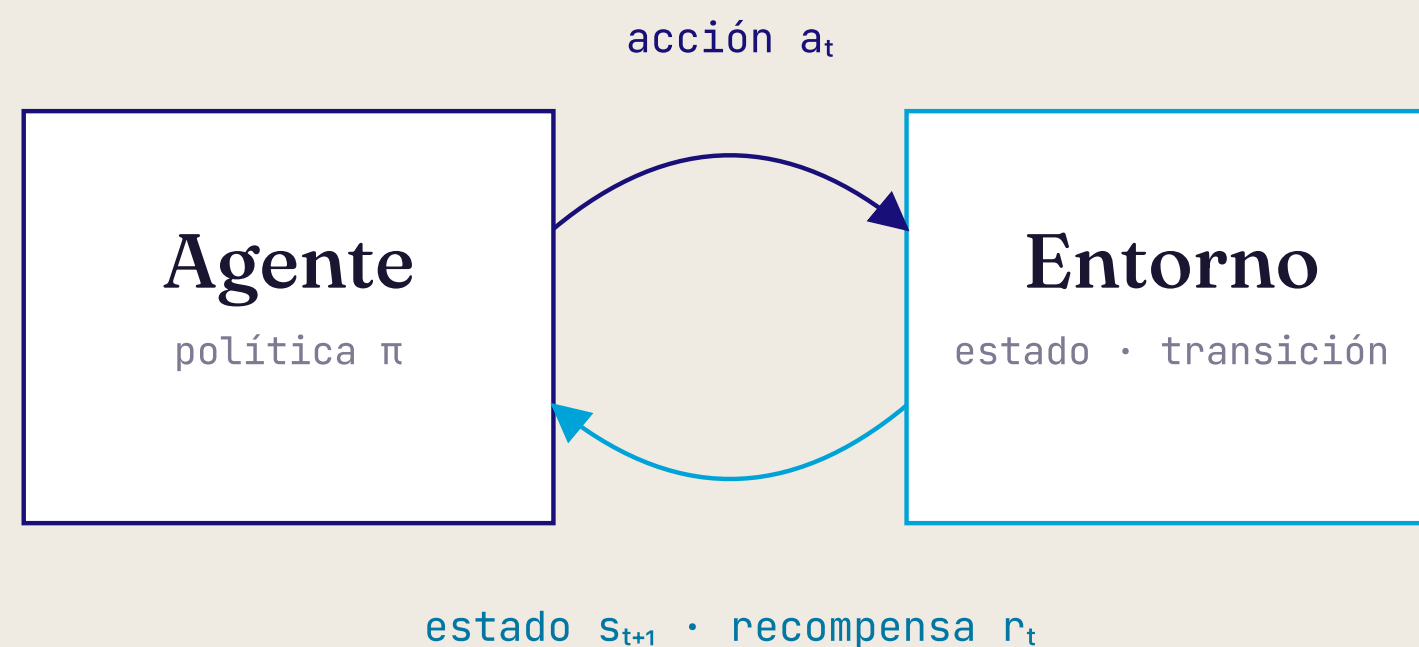
Etiquetar es caro; recoger datos crudos no tanto. El modelo aprovecha la *estructura* de los datos no etiquetados para mejorar la generalización a partir de pocos ejemplos etiquetados.

- Útil en medicina, voz, visión: anotar requiere experto.
- Self-training, pseudo-etiquetado, modelos de consistencia.
- Base de muchos modelos fundacionales actuales.

Aprender *actuando*.

Prueba, error y recompensa.

bucle de interacción



Un *agente* toma decisiones en un entorno. Cada acción cambia el estado y produce una **recompensa** (o castigo). El objetivo: maximizar la recompensa acumulada a largo plazo.

Formalmente, esto es un **Proceso de Decisión de Markov** (MDP) — estados, acciones, transiciones, recompensas y factor de descuento.

$\langle S, A, P, R, \gamma \rangle$

- Videojuegos, robótica, anuncios, conducción autónoma.
- Q-learning, policy gradients, actor-critic, RLHF.

¿Qué tipo de aprendizaje usaríais para *cada caso*?

#	PROBLEMA	TU RESPUESTA
A	Filtrar correos spam de la bandeja de entrada.	-----
B	Predecir cuántas unidades vamos a vender la semana que viene.	-----
C	Descubrir grupos naturales de clientes para campañas distintas.	-----
D	Enseñar a un robot a caminar en un simulador físico.	-----
E	Detectar tumores en radiografías con miles de imágenes anotadas.	-----
F	Estimar la edad de una persona a partir de una foto.	-----
G	Recomendar películas a millones de usuarios sin etiquetas explícitas.	-----
H	Entrenar a un agente para jugar al ajedrez compitiendo contra sí mismo.	-----

SUP · REGRESIÓN

SUP · CLASIFICACIÓN

NO SUPERVISADO

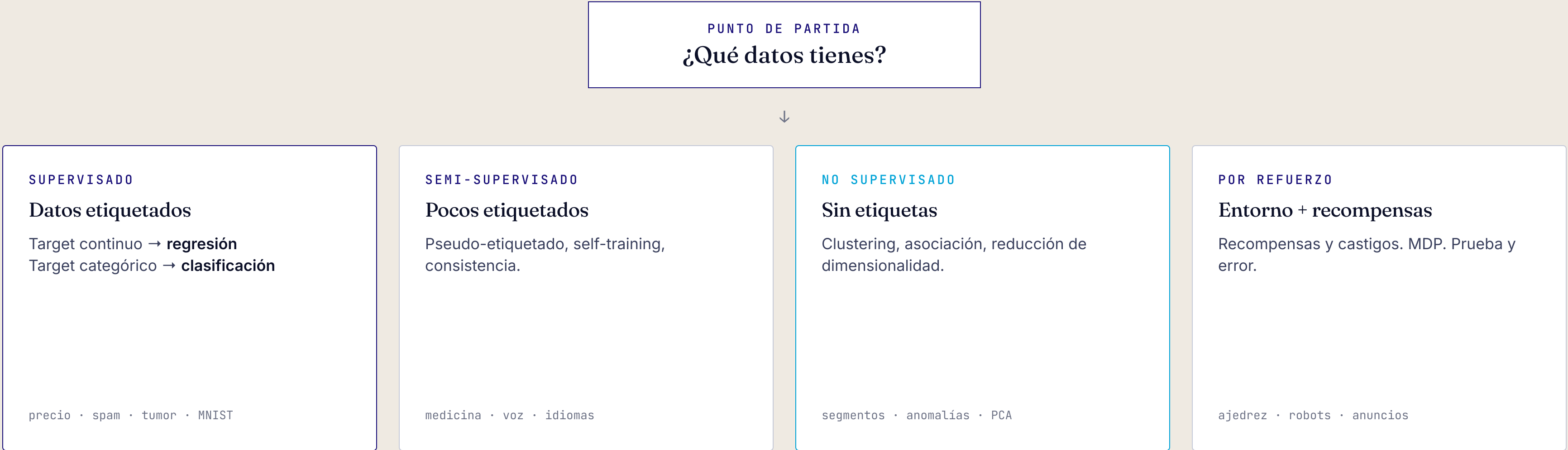
SEMI-SUPERVISADO

POR REFUERZO

► REVELAR SOLUCIÓN

RESUMEN VISUAL

Esquema diferenciador · cómo elegir el *tipo de aprendizaje*.



El tipo de aprendizaje no es el modelo: el mismo algoritmo puede entrenarse de varias maneras.

PRÓXIMA SESIÓN · ALGORITMOS EN DETALLE